

SE3110 — Software Testing & Quality Evaluation

Individual Student Viva Demonstration & Technical Defense Guide

Target System: WayPoint — Integrated Full-Stack & Agentic AI Transit Platform

Viva Overview & Rubric Breakdown (60 Individual Marks)

In the SE3110 assessment, 60% of the total marks are assessed individually during the viva demonstration:

Viva Assessment Criterion	Marks	What the Examiner Evaluates
Testing Tool / Framework Demonstration	15 Marks	Demonstrates tool execution live, explains selection, configuration, and integration with the actual system.
Test Implementation & Execution	15 Marks	Demonstrates meaningful personally authored tests across normal, invalid, boundary, and failure cases with strict assertions.
Results, Defects & Retesting	10 Marks	Explains test results, demonstrates defect root cause, walks through the source code fix, and proves retesting passes.
Technical Contribution & Traceability	5 Marks	Shows clear Git commits, file ownership, and architectural traceability to SE3090 system requirements.
Viva Technical Understanding & Live Modification	15 Marks	Confidently answers technical questions, explains design choices, and modifies a test live when challenged.
TOTAL INDIVIDUAL VIVA SCORE	60 Marks	Crucial component for the final module grade.

STUDENT 1: Vinranga M.W.S (IT24100374)

Role: Component 1 Owner — Journey Planning, Routes, Timetables & Journey Agent

1. Architectural Scope & Testing Responsibilities

- **Scope:** Intercity route catalogue, intermediate stops, distance offsets, timetables, journey search engine, connecting transfer window logic (BR-TRANSFER-001), and AI Journey Analysis Agent.
- **Testing Tools:** .NET 8 xUnit (JourneySearchControllerTests.cs), Vitest & RTL (RouteManager.test.jsx), Flutter Test (journey_search_test.dart), Pytest (test_journey_agent.py).

2. Live Viva Demonstration Commands

```
# 1. Demonstrate Backend Journey Search & Transfer Buffer Tests
dotnet test backend/WayPoint.sln --filter "FullyQualifiedName~JourneySearch"

# 2. Demonstrate Web Route Manager Component Tests
cd web && npx vitest run src/features/journey/__tests__/RouteManager.test.jsx

# 3. Demonstrate Mobile Journey Search Widget Tests
cd mobile && flutter test test/features/journey/journey_search_test.dart

# 4. Demonstrate AI Journey Agent Multi-Criteria Optimization
cd ai && pytest tests/test_journey_agent.py -v
```

3. Key Test Scenarios & Invariants to Explain

- **BR-TRANSFER-001 (Connecting Transfer Window):** Explains why connecting bus services require a minimum 20-minute gap. Shows the boundary test: 19-minute buffer fails; 20-minute buffer passes.
- **Normal Case:** Search between Colombo and Kandy returns direct express services and tourist corridor tags.
- **Boundary Case:** Connecting service departing exactly at $T_1 + 20\text{ min}$.
- **Invalid Case:** Origin and destination are identical; date is in the past.

4. Defect Discovered & Remediation Walkthrough (DEF-002)

- **Defect:** Connecting routes with unsafe 10-minute transfer gaps were recommended to passengers.
- **Root Cause:** Query used `DepartureTime > ArrivalTime` instead of enforcing `(DepartureTime - ArrivalTime).TotalMinutes >= 20.0`.
- **Code Fix:** Added LINQ filter in `WayPoint.Application/Services/JourneySearchService.cs`.
- **Retest Evidence:** `test_business_rule_connecting_transfer_buffer` passed in `tests/integration/test_cross_service_api.py`.

5. Likely Viva Questions & Ready Answers

- **Q:** Why did you select Vitest over Jest for the React RouteManager tests?
 - **A:** Vitest shares the exact same Vite pipeline and ES-module configuration as our React build, providing 10x faster HMR and native ESM execution without Babel transpilation overhead.
- **Q:** Can you modify your test right now to verify that a 15-minute transfer window is rejected?
 - **A:** Yes. In `test_business_rule_connecting_transfer_buffer`, adjust `itinerary_Leg2_departure` to 08:15:00 and assert `is_valid_transfer` evaluates to `False`.

Role: Component 2 Owner & Group Leader — Fleet, 2D Seat Maps, Driver Rostering & Resource Agent

1. Architectural Scope & Testing Responsibilities

- **Scope:** Bus fleet inventory, 2D visual seat layout designer (2+2, 1+2 layouts, aisle configurations), driver rosters, rest-hour compliance (`BR-TIME-001`), replacement bus feasibility, and AI Resource Feasibility Agent.
- **Testing Tools:** **.NET 8 xUnit** (`Fleet/DriverOverlapDetectionTests.cs` , `Fleet/ResourceFeasibilityTests.cs` , `Fleet/SeatLayoutValidationTests.cs`), **Vitest** (`SeatLayoutDesigner.test.jsx`), **Flutter Test** (`seat_picker_bloc_test.dart` , `review_submission_test.dart`), **Pytest** (`test_resource_agent.py`).

2. Live Viva Demonstration Commands

```
# 1. Demonstrate Fleet, Driver Rest & Seat Layout Backend Tests
dotnet test backend/WayPoint.sln --filter "FullyQualifiedName~Fleet"

# 2. Demonstrate Web 2D Seat Layout Designer Tests
cd web && npx vitest run src/features/fleet/__tests__/SeatLayoutDesigner.test.jsx

# 3. Demonstrate Mobile Seat Picker BLoC State Transitions
cd mobile && flutter test test/features/fleet/seat_picker_bloc_test.dart

# 4. Demonstrate AI Resource Feasibility & Driver Rest Validation
cd ai && pytest tests/test_resource_agent.py -v
```

3. Key Test Scenarios & Invariants to Explain

- **`BR-TIME-001` (Driver Rest Hours):** Explains the labor safety rule requiring >8 hours mandatory rest between intercity shifts.
- **2D Seat Matrix Generation:** Demonstrates mathematical coordinate generation ($R \times C$) with automatic aisle gap insertion.
- **Normal Case:** Assigning driver to Shift B starting 9 hours after Shift A completes.
- **Invalid / Failure Case:** Assigning driver to Shift B starting 5 hours after Shift A (fails with rest violation).
- **Boundary Case:** Shift gap of exactly 8 hours and 0 minutes.

4. Defect Discovered & Remediation Walkthrough (`DEF-006`)

- **Defect:** Driver roster scheduler accepted overlapping assignments without checking the 8-hour rest gap.

- **Root Cause:** Validation only checked for direct time interval overlaps (`StartA <= EndB && EndA >= StartB`).
- **Code Fix:** Added `restBufferHours = (newShift.StartTime - previousShift.EndTime).TotalHours;` `if (restBufferHours < 8.0) throw ...` in `DriverService.cs` .
- **Retest Evidence:** `DriverOverlapDetectionTests.cs` passed with 100% assertions green.

5. Likely Viva Questions & Ready Answers

- **Q:** *How did you test the interactive 2D Seat Picker on Flutter without running the emulator?*
 - **A:** Using `bloc_test` and `flutter_test` . We tested the `SeatPickerBloc` state machine directly: validating that `SelectSeat("12A")` emits `SeatsHolding` , starts a 600-second ticker, and rejects duplicate clicks when seats are held.
- **Q:** *What happens if an AI agent attempts to roster a driver who has only rested 6 hours?*
 - **A:** The `LangGraph` state transitions to `ResourceAgent` , but the deterministic Python guardrail `validate_driver_rest` intercepts the proposal and returns `is_valid: False` before any database record is written.

STUDENT 3: Dissanayaka A.D.M.N.K. (IT24100225)

Role: Component 3 Owner — Booking, Hold Concurrency, Payments & Digital Ticketing

1. Architectural Scope & Testing Responsibilities

- **Scope:** 10-minute temporary seat hold concurrency locks (`BR-HOLD-001`), payment sandbox gateway, cryptographic HMAC-SHA256 QR ticket issuance (`BR-HMAC-001`), tiered refunds (`BR-REFUND-001`), and AI Booking & Policy Agent.
- **Testing Tools:** **.NET 8 xUnit** (`BookingTests.cs`), **k6 & Python Benchmark Runner** (`run_concurrency_benchmarks.py`), **Vitest** (`BookingManifest.test.jsx`), **Flutter Test** (`ticket_wallet_test.dart`), **Pytest** (`test_booking_agent.py`).

2. Live Viva Demonstration Commands

```
# 1. Demonstrate Seat Hold Concurrency, HMAC Tampering & Tiered Refunds
dotnet test backend/WayPoint.sln --filter "FullyQualifiedName~BookingTests"

# 2. Demonstrate Live High-Concurrency Load Benchmark
python tests/performance/run_concurrency_benchmarks.py

# 3. Demonstrate Web Booking Manifest Monitor Tests
cd web && npx vitest run src/features/bookings/__tests__/BookingManifest.test.jsx

# 4. Demonstrate Mobile Ticket Wallet QR Code Tests
cd mobile && flutter test test/features/ticket_wallet_test.dart
```

3. Key Test Scenarios & Invariants to Explain

- **BR-HOLD-001 (Concurrency & Double-Booking Prevention):** Two concurrent requests fire simultaneously for Seat `12A`. Exactly one receives 200 OK and one receives 409 Conflict.
- **BR-REFUND-001 (Tiered Refund Schedule):**
 - Cancellation >24 h before departure: 90% refund.
 - Cancellation $12\text{--}24$ h before departure: 50% refund.
 - Cancellation <12 h before departure: 0% refund.
- **BR-HMAC-001 (Cryptographic Anti-Tamper QR Pass):** Generates HMAC-SHA256 hash using server secret; demonstrates that altering a single character in the payload causes immediate rejection upon conductor scan.

4. Defects Discovered & Remediation Walkthrough (`DEF-001` , `DEF-003` , `DEF-007`)

- **DEF-001 :** Concurrency race condition allowed simultaneous hold requests to both succeed. Remediation: Added EF Core optimistic concurrency tokens (`xmin`) and atomic transactions.
- **DEF-003 :** Floating-point precision bug in 50% refund calculation. Remediation: Converted all calculations to C# `decimal` with `Math.Round(..., MidpointRounding.AwayFromZero)` .
- **DEF-007 :** QR ticket payload only hashed booking reference, allowing attackers to forge seat numbers. Remediation: Updated HMAC to bind `WP|REF:...|SRV:...|SEATS:...|PASS:...|HMAC:... .`

5. Likely Viva Questions & Ready Answers

- **Q:** *How do you simulate race conditions deterministically in xUnit?*
 - **A:** In `BookingTests.cs` , we instantiate two separate `DbContext` instances pointing to the same database. We launch two controller tasks using `Task.WhenAll(task1, task2)` . We assert that the status codes contain exactly one 200 OK and one 409 Conflict.
- **Q:** *What were the P95 latency and throughput results from your k6/python benchmark?*
 - **A:** Under 25 concurrent workers, our seat hold endpoint achieved 12.35 requests/sec with a P50 median latency of 1924ms, successfully rejecting concurrent attempts without deadlocking.

STUDENT 4: Dineth sasmitha J.S.D. (IT24102912)

Role: Component 4 Owner — Disruption Mitigation, AI Safety, Approval Gate & Conductor Scanner

1. Architectural Scope & Testing Responsibilities

- **Scope:** Disruption incident intake, passenger blast radius analysis, multi-agent AI rebooking orchestration, Transport Manager approval gate (`BR-APPROVAL-001`), mobile conductor QR camera scanner, and Validation & Safety Agent.

- **Testing Tools:** **.NET 8 xUnit** (`DisruptionTests.cs`), **Pytest & LangGraph** (`test_safety_agent.py` , `test_safe_failure.py` , `test_tool_registry.py`), **Vitest** (`DisruptionHub.test.jsx`), **Flutter Test** (`conductor_tools_test.dart` , `disruption_alert_test.dart`).

2. Live Viva Demonstration Commands

```
# 1. Demonstrate Disruption Management, Approval State Machine & Audit Tests
dotnet test backend/WayPoint.sln --filter "FullyQualifiedName~DisruptionTests"

# 2. Demonstrate Closed-Loop Cross-Platform Integration Workflow (Fig. 2)
pytest tests/e2e/test_closed_loop_workflow.py -v

# 3. Demonstrate AI Tool Allow-List, Guardrails & Safe-Failure Tests
cd ai && pytest tests/test_safety_agent.py tests/test_safe_failure.py tests/test_tool_registry.py -v

# 4. Demonstrate Web Disruption Hub & Manager Approval Gate
cd web && npx vitest run src/features/disruptions/__tests__/DisruptionHub.test.jsx
```

3. Key Test Scenarios & Invariants to Explain

- **BR-APPROVAL-001 (Manager Approval Gate):** Explains why timetable delays > 15 minutes or route cancellations CANNOT be auto-executed and must wait in `PendingApproval` status for a human manager.
- **BR-AIT00L-001 (Strict 10-Tool Allow-List):** Demonstrates that the AI microservice rejects any tool outside the 10 allow-listed tools.
- **FR-AI-004 (Safe-Failure Circuit Breaker):** Demonstrates that 3 consecutive failures trip the circuit breaker and return a safe error object instead of crashing or retrying infinitely.
- **FR-AI-008 (Prompt Injection Defusing):** Demonstrates that jailbreak attempts in incident notes are stripped and quarantined.

4. Defects Discovered & Remediation Walkthrough (`DEF-004` , `DEF-005` , `DEF-008`)

- **DEF-004 :** Boundary condition allowed a 16-minute delay to execute without Manager Approval. Remediation: Corrected comparison from `>= 20` to `> 15` in `ApprovalService.cs` .
- **DEF-005 :** Prompt injection vulnerability in disruption description notes. Remediation: Built `sanitize_user_input` and `wrap_user_input` regex guardrails.
- **DEF-008 :** Indefinite retry loop on gateway outage causing microservice thread starvation. Remediation: Built `SafeFailureManager` with `MAX_RETRIES = 3` and short-circuit return.

5. Likely Viva Questions & Ready Answers

- **Q:** How does your testing prove that an AI agent cannot confirm a payment or modify a schedule on its own?
 - **A:** In `test_tool_registry.py` and `test_safety_agent.py` , we assert that neither `PaymentCharge` nor `DirectScheduleUpdate` exist in the 10 allow-listed tools. The safety agent

only possesses `create_rebooking_proposal` and `request_manager_approval`. Operational database mutations are only executed when a human manager sends an authenticated `POST /api/v1/approval/decide` request.

- **Q:** Can you run the closed-loop E2E test right now to prove that all tiers work together?
 - **A:** Yes. We run `pytest tests/e2e/test_closed_loop_workflow.py -v`. It validates the 7 steps: Health $\rightarrow$ Search $\rightarrow$ Hold $\rightarrow$ Disruption Intake $\rightarrow$ Manager Approval $\rightarrow$ Payment/QR $\rightarrow$ Conductor Verification in under 7 seconds.